



A novel semantic-aware search scheme based on BCI-tree index over encrypted cloud data

Qian Zhou¹ · Hua Dai^{2,3} · Yuanlong Liu² · Geng Yang^{2,3} · Xun Yi⁴ · Zheng Hu²

Received: 4 October 2022 / Revised: 7 April 2023 / Accepted: 14 April 2023 /

Published online: 31 May 2023

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract

Most of the traditional privacy-preserving search schemes in cloud adopt TF-IDF model which is on the basis of keyword frequency statistics. The embedding semantic association between keywords and documents are not considered. To solve this problem, we propose a novel semantic-aware search scheme based on BCI-tree index over encrypted cloud data. The LDA model is adopted to generate vectors for documents and queried keywords and the vectors contain topic-based semantic information. The homomorphic encryption on vectors is used to perform privacy-preserving semantic relevance score computation between queried keywords and documents. To achieve efficient search processing, a novel binary clustering tree-based index (BCI-tree index) is designed, which is constructed following the divisive hierarchical clustering algorithm. By using the BCI-tree index, a depth-first recursive search algorithm is proposed. In addition, a threshold presetting-based optimization is applied to further accelerate the search speed. The experimental results show that the proposed scheme performs good in the semantic precision of search results and the search time cost.

✉ Qian Zhou
zhouqian@njupt.edu.cn

Hua Dai
daihua@njupt.edu.cn

Yuanlong Liu
1021041407@njupt.edu.cn

Geng Yang
yangg@njupt.edu.cn

Xun Yi
xun.yi@rmit.edu.au

Zheng Hu
1219043732@njupt.edu.cn

¹ School of Modern Posts, Nanjing University of Posts and Telecommunications, 210023 Nanjing, China

² School of Computer Science, Nanjing University of Posts and Telecommunications, 210023 Nanjing, China

³ Jiangsu Key Laboratory of Big Data Security and Intelligent Processing, 210023 Nanjing, China

⁴ School of Computing Technologies, RMIT University, 3001 Melbourne, Australia

Keywords Cloud computing · Searchable encryption · BCI-tree index · Multi-keywords search · Semantic-aware search · Privacy preservation

1 Introduction

Nowadays, cloud computing is becoming more and more popular and useful technology for enterprises due to its X-as-service and pay-as-demand properties, which provides dynamic, easy-scalable and virtualized resources (storage, computation, etc.) through the Internet. Data and even software of enterprises could be outsourced and deployed in the cloud for cost saving. However, the outsourcing of data makes the data owner losing the independent and exclusive control on his/her data, which could lead privacy leakage issues. The cloud service provider could obtain the valuable information of the outsourced data by data snooping tools such as statistical interfering, etc. To protect data privacy, an easy idea is to encrypt the data before it is outsourced to the cloud. But the encrypted data is difficult to search and use, which severely decreases the availability of the data. It is a challenge to provide the searchability and the privacy preservation on the outsourced data, simultaneously. Therefore, it is a necessity to research on privacy-preserving search schemes.

Recently, many privacy-preserving search schemes have been proposed, such as single keyword search schemes [7, 9, 26, 32, 35, 38, 44] and multi-keyword search schemes [1, 3–6, 8, 10–12, 15, 19–22, 24, 25, 27–29, 33, 34, 36, 37, 39, 41, 43, 45–48]. Most schemes use the term frequency-inverse document frequency model (TF-IDF) based vector space model to generate vectors to represent documents and searched keywords, and these vectors are encrypted by the random invertible matrix based vector encryption method [4] for privacy preservation. The encrypted vectors are organized into the sophisticate encrypted indexes which support the privacy-preserving multi-keyword search processing in the cloud. But the TF-IDF model is a statistical model which utilizes the quantized frequency statistics to measure the importance of keywords to a collection of documents. It rarely considers the semantic association between keywords and documents, which could lead to unsatisfied results for users. For example, the queried keywords “apple, ios” indicates that the search intention is actually about the information of the operating system of apple phone. But under the frequency statistical based TF-IDF model, the returned result could have something about the fruit “apple”. Meanwhile, because of the large quantity of keywords extracted from the whole documents, the generated vectors following the TF-IDF model could have an extremely high dimension and be sparse. It will cost a lot of storage space and search time. The previous work in [10] firstly proposed a LDA-based semantic-aware search scheme, where documents are trained by the LDA model and represented by lower dimensional topic vectors. But the semantic relation between documents are not considered when constructing the search index, there is still room for improvement in search result precision and search processing efficiency.

In this paper, we proposed an novel **Efficient Privacy-preserving Semantic-Aware Search** scheme (EPSAS) over encrypted cloud data. The data owner first uses the LDA model to train documents and generate vectors for documents and the vectors contain topic-based semantic information. These vectors are taken as the input data of the divisive hierarchical clustering algorithm, and a novel binary clustering tree (BCI-tree) is formed. The BCI-tree is encrypted into secure index by the homomorphic matrix encryption, and the encrypted index is transmitted to the cloud server for outsourcing together with the encrypted documents. Once the data user starts a search having multiple queried keywords, it converts the queried keywords into a trapdoor for protecting the privacy of the queried keywords and then transmits

the trapdoor to the cloud server as the search request. The cloud server processes the search by making use of the secure index. After the data user receives the search result in ciphertext, it performs the decryption to obtain the plaintext result. To further improve the speed of searches, a threshold presetting-based optimization is presented. The experimental results show that the proposed scheme performs good in the semantic precision of search result and the search time cost.

The contributions of this paper are summarized as follows.

- 1) By adopting the LDA topic model and the divisive hierarchical clustering algorithm, we present a topic information-embedded binary clustering tree (BCI-tree) index. Based on the BCI-tree index, we present a depth-frist recursive search algorithm to achieve efficient privacy-preserving semantic-aware searches (EPSAS).
- 2) To further accelerate the search speed, we present a novel structure definition of threshold referring matrix (TR-matrix) which is used for proper pruning threshold determination. Based on TR-matrix, we propose a threshold presetting-based optimization on EPSAS.
- 3) To evaluate the proposed scheme, we conduct security analysis from the perspective of privacy preservation capabilities. We also perform experiments on a real dataset and the experimental result indicates that the proposed scheme are efficient and accurate.

We note that a shorter conference version of this paper appeared in [49]. The initial conference paper only presents the main procedures of the baseline search scheme. This journal paper significantly extents the conference paper in the following aspects: 1) we elaborate the baseline search scheme in detail, where the working principle of the pruning threshold is described and two intuitive examples are well designed to illustrate the index and search algorithm; 2) we propose a newly designed threshold presetting-based optimization on the baseline search scheme, which can further accelerate the search speed; 3) we add security analysis to describe the security of the proposed search scheme; 4) we enrich experiments to evaluate the baseline version and optimized version of the proposed search scheme. The result shows the efficiency of the optimized search scheme.

The remainder of our paper is organized as follows. Section 2 presents notations and preliminaries. Section 3 introduces the system model, threat model, semantic-aware search model, design goals and framework of the proposed scheme. Section 4 gives the baseline search scheme while Section 5 presents an optimized version. Sections 6 and 7 present security analyses and performance evaluation of the proposed scheme. Section 8 concludes this paper.

2 Notations and preliminaries

In this section, we will introduce some basic Notations and Preliminaries.

2.1 Notations

- F : a set of n documents, $F = \{f_1, f_2, \dots, f_n\}$.
- $\widehat{F}$: the encrypted document set of F , $\widehat{F} = \{\widehat{f}_1, \widehat{f}_2, \dots, \widehat{f}_n\}$.
- D : a dictionary with h keywords, $D = \{w_1, w_2, \dots, w_h\}$, which is extracted from documents of F .
- T : a set of m topics, $T = \{t_1, t_2, \dots, t_m\}$.
- Γ : An $n \times m$ -dimensional document-topic correlation matrix.
- $\widehat{\Gamma}$: the encrypted matrix of Γ .

- Ψ : an $h \times m$ -dimensional keyword-topic correlation matrix.
- P_T : an m -dimensional vector which stores topic probabilities.
- P_W : an h -dimensional vector which stores keyword probabilities.
- $\mathcal{I}$: a binary tree-based index.
- $\widehat{\mathcal{I}}$: the encrypted index of $\mathcal{I}$.
- Q : a search with multiple interested keywords, $Q \subseteq D$.
- V_Q : an m -dimensional query topic vector.
- $\widehat{V}_Q$: the encrypted vector of V_Q .
- τ : a threshold used for filtering subtrees.

2.2 Preliminaries

We introduce the preliminaries of the LDA model, the semantic-aware relevance measurement, the divisive hierarchical clustering, the Hadamard product, and the secure inner product, which are adopted in this paper to achieve the semantic-aware searches over encrypted cloud data.

LDA Model The Latent Dirichlet Allocation model (LDA) is an unsupervised learning algorithm proposed by David Blei [2]. The LDA model takes topics as the associations between keywords and documents. It views each document as a mixture of implicit topics, and each topic is represented by a probability distribution of terms related to the topic. The LDA model can be utilized to perceive potential topic information in a large corpus.

By using the LDA model to train the documents of F , a document-topic correlation matrix Γ and a keyword-topic correlation matrix Ψ will be generated, which describe the numerical topic relevance of documents and keywords to topics, respectively. Each document is represented by a document topic vector while and each keyword is represented by a keyword topic vector, which are described as follows.

- **Document Topic Vector** Given a document f_i and a topic t_j , $\Gamma[i]$ is the document topic vector of f_i , where $\Gamma[i][j]$ records the topic relevance score between f_i and t_j .
- **Keyword Topic Vector** Given a keyword w_i and a topic t_j , $\Psi[i]$ is the keyword topic vector of w_i , where $\Psi[i][j]$ records the topic relevance score between w_i and t_j .

LDA-based semantic relevance measurement In this paper, by using LDA model, documents and queried keywords are demonstrated by the vectors embedding the topic semantic information. The semantic relevance score between a document and the queried keywords is measured by the inner product between the corresponding vectors, which is similar to the work [40]. Assuming that $\Gamma[i]$ is the document topic vector of the document f_i and V_Q is the query topic vector of the search Q , the semantic relevance score between f_i and Q is described in (1).

$$\text{Score}(f_i, Q) = \Gamma[i] \cdot V_Q \quad (1)$$

Divisive hierarchical clustering The divisive hierarchical clustering algorithm [30] is one of the hierarchical clustering algorithms, which can divide a large cluster into multiple small sub-clusters recursively. Specifically, all samples are initially classified into a cluster. Then, the distance between any two samples is calculated in the same cluster and the two samples a and b with the farthest distance are found. The samples a and b are assigned to the newly created sub-clusters C_1 and C_2 , respectively. Each sample x of the rest is then assigned into C_1 or C_2 according to the distance between the sample x and a and b . If the distance between

x and a is farther than that between x and b , the sample x is classified into C_1 , otherwise it is classified into C_2 . The above clustering process is repeated until a certain condition is satisfied. The top-down clustering algorithm will form a binary clustering tree. Each node in the tree represents a cluster and its left child node and right child node represent the generated two sub-clusters.

Hadamard product Given two vectors $A = (a_1, a_2, \dots, a_n)$ and $B = (b_1, b_2, \dots, b_n)$, $A \circ B$ is the Hadamard product between A and B , where “ $\circ$ ” is the Hadamard product operator. The calculation is presented in (2).

$$A \circ B = (a_1 b_1, a_2 b_2, \dots, a_n b_n) \quad (2)$$

According to (2), the calculation result of Hadamard product is also an n -dimensional vector.

Secure inner product The secure inner product [4] is implemented by an invertible matrix-based homomorphic encryption. Given two n -dimensional vectors A and B , and a random generated $n \times n$ invertible matrix M , A and B are encrypted by M as shown in (3) and the encrypted result vectors are denoted as $\hat{A}$ and $\hat{B}$, respectively.

$$\begin{cases} \hat{A} = M^T A \\ \hat{B} = M^{-1} B \end{cases} \quad (3)$$

The inner product between A and B is equal to the inner product between $\hat{A}$ and $\hat{B}$, which is proved by (4). It means that the inner product of two vectors can be calculated even just knowing the corresponding encrypted vectors, and the plaintext vectors are not the must.

$$\hat{A} \cdot \hat{B} = (M^T A) \cdot (M^{-1} B) = (M^T A)^T (M^{-1} B) = A^T M M^{-1} B = A \cdot B \quad (4)$$

3 Models and problem description

3.1 System model

In this paper, we adopt the same system model as that in [4, 5, 22, 34, 41, 47], which is shown in Figure 1. There are three entities in this system model, which are the data owner, the data user and the cloud server. They cooperate as follows to setup or perform the search processing.

- The data owner owns a set of documents. It uses Symmetric encryption algorithm to encrypt the documents and outsources them to the cloud server together with the encrypted index. It shares private keys and other information with the data user for processing searches.
- The cloud server provides storage for storing the encrypted documents and indexes outsourced by the data owner. When receiving a search trapdoor from the data user, the cloud server starts the search processing by applying the encrypted index and finally transmits k encrypted documents to the data user as the search result.
- The data user has the authorization for using the data owner's data. When the data user starts a search having multiple queried keywords to request k encrypted documents with the k highest semantic relevance scores to the search, it converts the queried keywords into a search trapdoor and then submits the trapdoor to the cloud server as the search request. When receiving the encrypted result documents from the cloud server, the data user performs decryption to get the search result in plaintext.

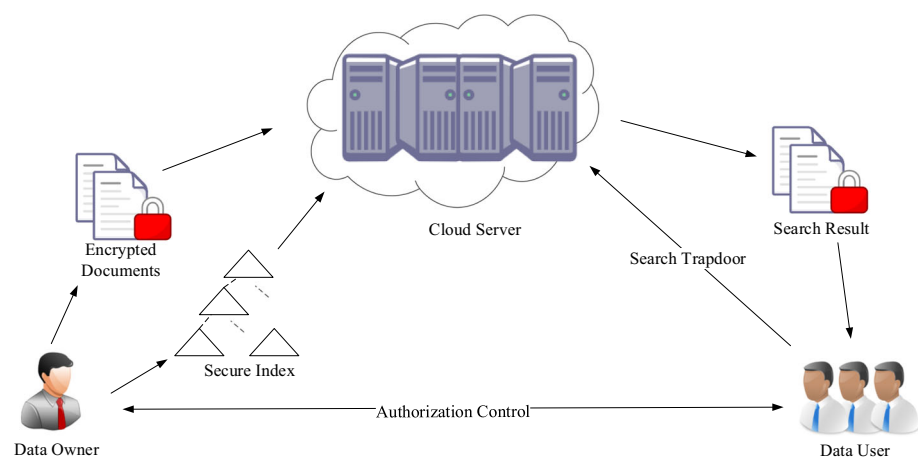


Figure 1 The system model

3.2 Threat model

The same threat model as the existing works [4, 5, 8, 10, 34, 37–39, 41, 43] is adopted in this paper, where the data owner and the data user are trust but the cloud server is honest-but-curious. Specifically, the cloud server performs searches following the pre-established protocols and returns the correct results to the data user, honestly. However, the cloud server is curious about the information of the outsourced data and it could snoop the information by statistical analyzing, inferring or other methods. We adopt the following known ciphertext threat model and known background threat model [4, 5, 8, 10, 34, 37–39, 41, 43] according to the background information that the cloud server knows.

Known ciphertext threat model In this model, the cloud server only knows the encrypted documents in $\hat{F}$, the secure index $\hat{\mathcal{I}}$ and the search trapdoor $\hat{V}_Q$, and it could perform the ciphertext-only attack (COA) [13] in this model.

Known background threat model In this stronger model, the cloud server can obtain more information than the known ciphertext threat model, such as the term frequency statistics of documents. The statistical information indicates the distribution of term frequency of

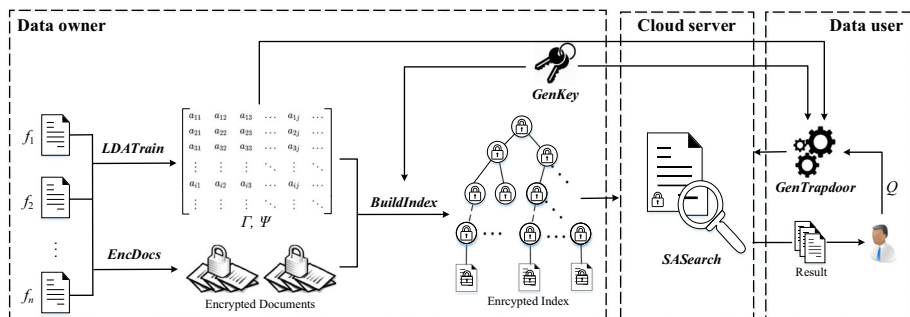


Figure 2 The framework of EPSAS

specific keywords in documents. The knowledge could be abused by the cloud server to perform statistical attacks to deduce or even identify specific keywords by analyzing the histogram or value range of the frequency distributions [38, 41].

3.3 Semantic-aware search model

A semantic-aware search is to get k documents from F , which has the k highest semantic relevance scores to the queried keywords. Let R be the search result of Q , (5) holds.

$$\begin{cases} |R| = k \\ \forall f_i \in R, f_j \in (D - R) \rightarrow \Gamma[i] \cdot V_Q > \Gamma[j] \cdot V_Q \end{cases} \quad (5)$$

where $\Gamma[i]$ and $\Gamma[j]$ are the document topic vectors of f_i and f_j , respectively, and V_Q is the query topic vector of Q . In general, the quantity of requested documents is much smaller than the quantity of documents in total, i.e., $k \ll n$.

3.4 Design Goals

Given a multi-keyword search Q , the design goals of the proposed scheme is described as follows.

Multi-keyword search In the scheme, the cloud server can return a list of k documents as the search result, which have the k highest semantic relevance scores to Q .

Search efficiency and accuracy The novel scheme can achieve efficient and accurate semantic-aware multi-keyword searches by using the LDA topic model, the novel BCI-tree index and search processing algorithm.

Privacy preservation The scheme can preserve the privacy of the outsourced data from the curious cloud server. The specific privacy-preserving requirements are shown as follows.

- *Document privacy*: The plaintext of documents should be protected from the curious cloud server.
- *Index privacy and search privacy*: The document topic vectors stored in the index and the queried keywords of a search should be protected.
- *Trapdoor unlinkability* [5, 8, 39, 41]: The cloud server can not confirm whether two trapdoors are corresponding to the same search.
- *Keyword privacy*: The cloud server should not identify any specific keyword of a document or a trapdoor.
- *Topic privacy*: In the LDA model, topics reflect the relationship between documents and keywords from some latent perspectives. The topic information of documents and keywords should be protected.

3.5 Framework of EPSAS

To overview the proposed scheme EPSAS, we present the framework as shown in Figure 2. It consists of three modules, the data preprocessing module, the index building module and the semantic-aware search module. The first module is to preprocess data to install the search system, which consists of three algorithms, *GenKey*, *LDATrain* and *EncDocs*. The second module is to construct an encrypted index for efficient search, which has the algorithm *BuildIndex*. The last module is to implement multi-keyword searches by the cooperations

between the data user and the cloud server. The brief introduction about these key algorithms are presented as follows:

- *GenKey*. The data owner uses this algorithm to generate the secure key K .
- *LDATrain*. The data owner uses this algorithm to extract keywords from F and then perform Gibbs sampling to generate the document-topic correlation matrix Γ and the keyword-topic correlation matrix Ψ .
- *EncDocs*. The data owner uses this algorithm to encrypt documents of F into $\hat{F}$ by the symmetric encryption.
- *BuildIndex*. The data owner uses this algorithm to construct an encrypted index $\hat{\mathcal{I}}$.
- *GenTrapdoor*. The data user uses this algorithm to generate trapdoors for multi-keyword searches, which are transmitted to the cloud server as the search commands.
- *SASearch*. The cloud server uses this algorithm to perform semantic-aware searches over encrypted cloud data and returns the search results.

4 The baseline search scheme

In this section, we present the baseline version of the efficient privacy-preserving semantic-aware search scheme (EPSAS), which uses a new designed BCI-tree index. There are three modules in the scheme, which are the data preprocessing module, index building module and semantic-aware search module, which are introduced as follows to demonstrate the idea of EPSAS.

4.1 Data preprocessing module

The data preprocessing module mainly includes the private key generation, the LDA model training and the document encryption, which are described as the algorithms *GenKey*, *LDATrain* and *EncDocs* as follows. These algorithms are all executed by the data owner.

$K \leftarrow \text{GenKey}(\mathbf{1}^{l(n)})$ The *GenKey* algorithm is to generate a set of private keys $K = \{S, M_1, M_2, k_f\}$, where S is an m -bit random vector, M_1 and M_2 are two random $m \times m$ invertible matrices, and k_f is the key used to encrypt documents. K is only shared between the data owner and the authorized data user but the cloud server has no idea of it.

$\{\Gamma, \Psi, P_T, P_W\} \leftarrow \text{LDATrain}(F)$ The *LDATrain* algorithm is to generate the document-topic correlation matrix Γ , the keyword-topic correlation matrix Ψ , the topic probability vector P_T and the keyword probability vector P_W . The algorithm is shown as the following steps.

- 1) Taking the documents of F as the input, the data owner uses the LDA model to perform the train processing, and the document-topic correlation matrix Γ and the keyword-topic correlation matrix Ψ are generated.
- 2) The data owner computes the topic probability vector P_T , the i th dimension of which stores the prior belief of the topic t_i existing in F . Assuming that documents in F have the same importance, the prior probability of t_i is the average of the probabilities that each document of F belongs to t_i . The calculation of P_T is presented in (6).

$$P_T = \frac{1}{n} \cdot \sum_{j=1}^n \Gamma[j]. \quad (6)$$

- 3) The data owner computes the keyword probability vector P_W , the i th dimension of which stores the prior belief of the keyword w_i existing in F . The calculation of P_W is presented in (7) which depends on P_T and Ψ .

$$P_W = P_T \cdot \Psi^T. \quad (7)$$

- 4) The data owner shares K , Ψ , P_T and P_W generated in the above steps to the data user.

$\hat{F} \leftarrow \text{EncDocs}(F)$ The *EncDocs* algorithm is to encrypt the documents of F by the symmetric encryption (AES, DES, etc.) with the generated key k_f of K . The generated encrypted documents of $\hat{F}$ are then transmitted to the cloud server for data outsourcing.

4.2 Index building module

This section proposes the structure definition and the construction algorithm of a novel binary clustering tree based index which is denoted as the BCI-tree.

Definition 1 Maximized Vector Let H be a set of m -dimensional vectors, the maximized vector of H is denoted as $\text{MaxVec}(H)$ which is an m -dimensional vector. The i th dimension of $\text{MaxVec}(H)$ is calculated according to (8),

$$\text{MaxVec}(H)[i] = \max\{V_j[i] \mid V_j \in H\}, \quad (8)$$

where $\max\{*\}$ is to get the maximum of an item set.

Definition 2 BCI-tree The BCI-tree is a binary tree which is constructed by the divisive hierarchical clustering algorithm. The tree is used as the search index and denoted as $\mathcal{I}$. Each node e of $\mathcal{I}$ is represented by a four-element tuple, $e = \langle id, lp, rp, vector \rangle$, where id is an identity, lp and rp respectively point to the left and right child nodes of e , and $vector$ is an m -dimensional vector. The setup of e depends on the position that e exists in $\mathcal{I}$.

- If e is a leaf node, then e represents a document. We assume the corresponding document is f_i . The setups of e are $e.lp = e.rp = \emptyset$, $e.id = i$ and $e.vector = \Gamma[i]$, where $e.id = i$ represents the identity of f_i .
- If e is not a leaf node, then $e.lp$ and $e.rp$ respectively point to the left and right child nodes of e , $e.id = \emptyset$, and $e.vector$ is the maximized vector of $e.lp$ and $e.rp$ where $e.vector = \text{MaxVec}(\{e.lp.vector, e.rp.vector\})$.

Lemma 1 Given a BCI-tree $\mathcal{I}$, an m -dimensional vector V and a parameter τ , if the inner product between V and the vector of the root node of $\mathcal{I}$ is smaller than τ , then the inner product between V and the vector of any node of $\mathcal{I}$ must be smaller than τ , i.e.,

$$V \cdot \mathcal{I}.root.vector < \tau \rightarrow \forall u \in \mathcal{I} (V \cdot u.vector < \tau), \quad (9)$$

where $\mathcal{I}.root$ is the root node of $\mathcal{I}$.

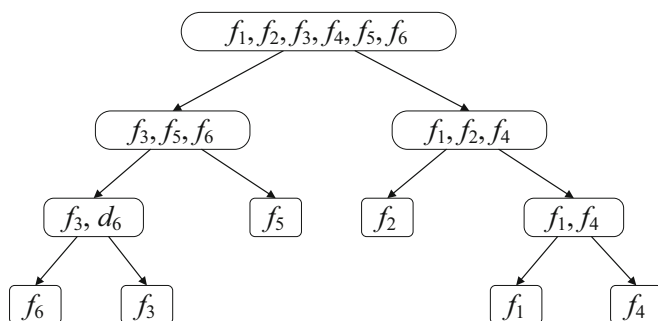
Proof Given an internal node u of $\mathcal{I}$, according to the structure definition of BCI-tree shown in Definition 2, the vector of the node u is the maximized vector of the vectors stored in the left and right child nodes of u , i.e., $u.vector = \text{MaxVec}(u.lp.vector, u.rp.vector)$. According to Definition 1, we have $u.vector[i] \geq u.lp.vector[i]$ and $u.vector[i] \geq u.rp.vector[i]$, where $i \in \{1, 2, \dots, m\}$. It means that the i th dimension of a node's vector is not smaller than that of its child nodes' vectors, and the relation is transitive. Thus, for the root of $\mathcal{I}$, the i th dimension of the root's vector is not smaller than that of any node's vectors. Therefore, if $V \cdot \mathcal{I}.root.vector < \tau$, then we have $\forall u \in \mathcal{I} (V \cdot u.vector < \tau)$ and Lemma 1 is proved. $\square$

Algorithm 1 *GenBCItree*(C).**Require:** C is a cluster of document topic vectors representing a set of corresponding documents**Ensure:** the root of a BCItree

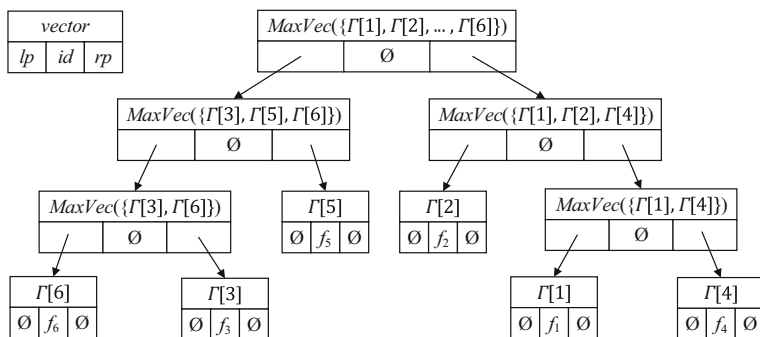
```

1: Create a new BCItree node  $u$ ;
2: if  $|C| > 1$  then
3:    $u.id = \emptyset$ ;
4:    $u.vector = \text{MaxVec}(C)$ ;
5:   Take the cluster  $C$  as the input to run the divisive hierarchical clustering algorithm, and the output
   sub-clusters are assumed to be  $C_1$  and  $C_2$ ;
6:    $u.lp = \text{GenBCItree}(C_1)$ ;
7:    $u.rp = \text{GenBCItree}(C_2)$ ;
8: else if  $|C| = 1$  then
9:   Assume that the only one document topic vector in  $C$  is  $\Gamma[i]$  which corresponds to the document  $f_i$ ;
10:   $u.vector = \Gamma[i]$ ;
11:   $u.id = i$ ;
12:   $u.lp = u.rp = \emptyset$ ;
13: end if

```



(a) The binary clustering tree



(b) The BCI-tree

Figure 3 An example of BCI-tree construction

The BCI-tree can be used as the index for efficient search processing, and Lemma 1 can be used to further improve search efficiency where the usage of the parameter τ is the key. The idea of search processing by using the BCI-index and Lemma 1 will be introduced in the semantic-aware search module in the next subsection.

$\hat{\mathcal{I}} \leftarrow \text{BuildIndex}(\Gamma, K)$ The data owner first takes the document topic vectors of $\Gamma = \{\Gamma[1], \Gamma[2], \dots, \Gamma[n]\}$ as the input and uses the divisive hierarchical clustering algorithm to construct the plaintext BCI-tree index $\mathcal{I}$. Then, the data owner encrypts the index with the private keys of K . The details of the *BuildIndex* algorithm is shown as follows:

- 1) Taking the document topic vectors of Γ as input, the data owner performs the *GenBCI-tree* algorithm to construct the plaintext BCI-tree index $\mathcal{I}$. The detail of *GenBCI-tree* is presented in Algorithm 1.
- 2) For each node in $\mathcal{I}$, the data owner splits the vector V stored in the node into two random vectors, V' and V'' , according to the m -bit random vector S in K . The split rule follows (10).

$$\begin{cases} V'[i] = V''[i] = V[i], & S[i] = 0 \\ V'[i] + V''[i] = V[i], & S[i] = 1 \end{cases} \quad (10)$$

Then, the data owner uses the random invertible matrices M_1 and M_2 in K to encrypt $\{V', V''\}$ into $\{\hat{V}', \hat{V}''\}$, respectively. The encryption follows (11).

$$\hat{V} = \{\hat{V}', \hat{V}''\} = \{V' M_1^{-1}, V'' M_2^{-1}\} \quad (11)$$

When the vectors of all nodes in $\mathcal{I}$ have been processed, the encrypted index $\hat{\mathcal{I}}$ is generated and then transmitted to the cloud server as the privacy-preserving search engine.

We give an example to illustrate the BCI-tree as shown in Figure 2. Given a document set $F = \{f_1, f_2, \dots, f_6\}$ whose document topic vectors are $\Gamma = \{\Gamma[1], \Gamma[2], \dots, \Gamma[6]\}$, respectively. Assuming that Figure 3(i) is the binary clustering tree generated by the divisive hierarchical clustering algorithm, the plaintext BCI-tree generated by Algorithm 1 is shown in Figure 3(ii), where the leaf nodes store the document topic vectors while the intermediate nodes store the maximized vectors.

4.3 Semantic-aware search module

In the semantic-aware search module, when the data user starts a search having multiple queried keywords, it first converts the queried keyword into a trapdoor and then transmits the trapdoor to the cloud server as the search request. Once receiving the search trapdoor, the cloud server performs the semantic-aware search by using the encrypted index and returns k encrypted documents as the result. When the data user receives the encrypted search result, it will obtain the plaintext result after decryption with the shared private key. There are two key algorithms in the semantic-aware search module, *GenTrapdoor* and *SASearch*. The former is to generate the trapdoor corresponding to a search with multiple queried keywords and the latter is to perform the search processing in privacy-preserving manner by calling a depth-first search algorithm.

$\hat{V}_Q \leftarrow \text{GenTrapdoor}(Q, \Psi, P_T, P_W, K)$ We assume that the started search is $Q = \{w_{e_1}, w_{e_2}, \dots, w_{e_p}\}$, where w_{e_p} is the e_p th keyword in D , and the search requests k documents which have the k highest semantic relevance scores to Q . The data user first generates a query topic vector V_Q and then uses the private keys of K to encrypt V_Q into the trapdoor $\hat{V}_Q$. The generation of a trapdoor is presented in detail as follows.

- 1) The data user generates the query topic vector V_Q according to (12) [40], where $\circ$ is the Hadamard product operator. V_Q is an m -dimensional vector that represents the probabilities that the topic intent of the search Q belongs to the topics of T .

$$V_Q = \sum_{w_{e_i} \in Q} \frac{P_T \circ \Psi[e_i]}{|Q| \cdot P_W[e_i]}, \quad (12)$$

- 2) The data user splits the vector V_Q into two random vectors $\{V'_Q, V''_Q\}$ with the n -bit random vector S in K . The split rule is shown in (13).

$$\begin{cases} V'_Q[i] + V''_Q[i] = V_Q[i] & S[i] = 0 \\ V'_Q[i] = V''_Q[i] = V_Q[i] & S[i] = 1 \end{cases} \quad (13)$$

- 3) The data user encrypts $V_Q = \{V'_Q, V''_Q\}$ into $\widehat{V}_Q = \{\widehat{V}'_Q, \widehat{V}''_Q\}$ by using the random inverted matrices M_1 and M_2 in K according to (14).

$$\widehat{V}_Q = \{\widehat{V}'_Q, \widehat{V}''_Q\} = \{V'_Q M_1^T, V''_Q M_2^T\} \quad (14)$$

The generated encrypted query topic vector $\widehat{V}_Q$ is the trapdoor which is transmitted to the cloud server as the search command.

According to the generation principles of the encrypted vectors of BCI-tree nodes and the trapdoors, we can deduce Lemma 2 as follows.

Lemma 2 *Given a vector stored in a node of a BCI-tree $\widehat{V}_i$ and a trapdoor $\widehat{V}_Q$, the plaintext vectors of which are V_i and V_Q , we have*

$$\widehat{V}_i \cdot \widehat{V}_Q = V_i \cdot V_Q. \quad (15)$$

Proof According to the algorithms *BuildIndex* and *GenTrapdoor*, $\widehat{V}_i$ and $\widehat{V}_Q$ are both encrypted vectors where $\widehat{V}_i = \{\widehat{V}'_i, \widehat{V}''_i\} = \{V'_i M_1^{-1}, V''_i M_2^{-1}\}$ and $\widehat{V}_Q = \{\widehat{V}'_Q, \widehat{V}''_Q\} = \{V'_Q M_1^T, V''_Q M_2^T\}$. The computation of the inner product between $\widehat{V}_i$ and $\widehat{V}_Q$ is shown as follows:

$$\begin{aligned} \widehat{V}_i \cdot \widehat{V}_Q &= (V'_i M_1^{-1})(V'_Q M_1^T)^T + (V''_i M_2^{-1})(V''_Q M_2^T)^T \\ &= V'_i \cdot V'_Q + V''_i \cdot V''_Q \\ &= V_i \cdot V_Q. \end{aligned}$$

Therefore, Lemma 2 holds. $\square$

Lemma 2 indicates that the inner product between a vector stored in a node of a BCI-tree index and trapdoor equals to the inner product between the corresponding plaintext vectors. Thus, the cloud server can perform secure inner production computation even without knowing the plaintext information of vectors.

$R \leftarrow \text{SASearch}(\widehat{\mathcal{T}}, \widehat{V}_Q, k)$ Once receiving a search trapdoor $\widehat{V}_Q$, the cloud server performs the *SASearch* algorithm by using the encrypted index $\widehat{\mathcal{T}}$ to efficiently obtain the search result R . To describe the idea of efficient search processing, we first present the definition of pruning threshold and then we propose a threshold-based filtering lemma which is capable of filtering out those unqualified sub-trees.

Definition 3 Pruning Threshold In the search processing with a BCI-tree index, a pruning threshold, denoted as τ , is a parameter for filtering the unqualified sub-trees which are the sub-trees that definitely do not contain any search results in their leaf nodes.

We present the value setting of τ in the search processing. Given a top- k search Q with the trapdoor $\widehat{V}_Q$, when a list of leaf nodes have been traversed and checked, the pruning threshold τ is set according to (16),

$$\tau = \begin{cases} 0 & |L| < k \\ \min\{u.\text{vector} \cdot \widehat{V}_Q | u \in C\} & |L| \geq k \end{cases} \quad (16)$$

where L is a list storing the traversed leaf nodes, and

$$|C| = k \wedge \forall x \in C, y \in (L - C) \rightarrow x.\text{vector} \cdot \widehat{V}_Q > y.\text{vector} \cdot \widehat{V}_Q.$$

As shown in (16), if the number of traversed leaf nodes $|L|$ is less than the number of requested documents k , then τ should be set to 0; otherwise τ should be set to the lower bound (or minimum) of the top- k inner products between the trapdoor vector and the encrypted document vectors in the traversed leaf nodes. The set C contains k leaf nodes corresponding to the top- k inner products and the documents in the leaf nodes of C are the candidate search result.

Lemma 3 *Given a trapdoor $\widehat{V}_Q$, when traversing nodes in the BCI-tree index $\widehat{\mathcal{I}}$ for obtaining the search result, for a node u , if $u.\text{vector} \cdot \widehat{V}_Q < \tau$ holds, then the documents in the leaf nodes of the sub-tree rooted by u are not in the search result.*

Proof For simply description, we assume that the sub-tree rooted by u is denoted as $\widehat{\mathcal{I}}_u$. According to Lemma 2, if $u.\text{vector} \cdot \widehat{V}_Q < \tau$, then we can deduce $\forall x \in \widehat{\mathcal{I}}_u \rightarrow x.\text{vector} \cdot \widehat{V}_Q < \tau$. It means that the inner products between the trapdoor vector and the encrypted document vectors in the leaf nodes of $\widehat{\mathcal{I}}_u$ are all smaller than τ . According to (16), the pruning threshold τ is the lower bound of the inner products between the trapdoor vector and the encrypted document vectors of candidate search result, thus the documents corresponding to the leaf nodes of $\widehat{\mathcal{I}}_u$ have no chance to be candidates, which means that they are not in the search result. Therefore, Lemma 3 holds. $\square$

Lemma 3 indicates that in the search processing, when a node of the BCI-tree index is traversed and checked, if the inner product between the trapdoor vector and the vector in the node is smaller than the pruning threshold τ , then the documents corresponding to the leaf nodes of the sub-tree rooted by the node are definitely not in the search result. Thus, there is no need to check any nodes in the sub-tree and we name this kind of sub-trees are the unqualified sub-trees. The unqualified sub-trees can be directly filtered out for accelerating search processing.

By adopting the BCI-tree index and Lemma 3, we present the search processing algorithm *SASearch* which calls a recursive depth-first search algorithm *DepthFirstSearch*. These two algorithms are shown in Algorithm 2 and 3.

As shown in Algorithm 2 and 3, a multi-keyword search is implemented in the manner of recursive depth-first processing. The pruning threshold τ is used to filtering unqualified sub-trees and accelerate search processing, which is initialized to be 0 at the beginning. As more leaf nodes are traversed, τ increases gradually, the unqualified sub-tree pruning effect is better, and the search processing speeds up.

We give an example to illustrate the search processing as shown in Figure 4. Assuming that the search trapdoor is $\widehat{V}_Q = [0.8, 0, 0, 0.5]$ and the number of requested documents is $k=2$. According to Algorithm 2 and 3, the search processing starts from the root node $u_{1,1}$, gets to the leaf node $u_{4,1}$ through the red path, and then gets to the leaf node $u_{4,2}$ through the blue path. At this time, the pruning threshold is updated where $\tau = u_{4,1}.\text{vector} \cdot \widehat{V}_Q = 0.05$,

Algorithm 2 $SASearch(\widehat{\mathcal{I}}, \widehat{V}_Q, k)$.**Require:** $\widehat{\mathcal{I}}$ is the BCI-tree index, $\widehat{V}_Q$ is the search trapdoor, and k is the number of requested documents**Ensure:** the search result R

```

1: Initial the pruning threshold  $\tau = 0$ , the leaf node set  $NS = \emptyset$ , and the search result  $R = \emptyset$ ;
2: Run the algorithm  $DepthFirstSearch(\widehat{\mathcal{I}}.root, \widehat{V}_Q, k, \tau, NS)$ , where  $\widehat{\mathcal{I}}.root$  represents the root node of  $\widehat{\mathcal{I}}$ ;
3: for each leaf node  $u \in NS$  do
4:   Get the encrypted document according to the identity  $u.id$ , and then add it to  $R$ ;
5: end for
6: return  $R$ ;

```

Algorithm 3 $DepthFirstSearch(u, \widehat{V}_Q, k, \tau, NS)$.**Require:** u is the root of a sub-BCI tree, $\widehat{V}_Q$ is the search trapdoor, k is the number of requested documents, τ is the pruning threshold, and NS is the set of leaf nodes storing candidate search results**Ensure:** the set of leaf nodes NS storing candidate search result

```

1: if  $u \neq \emptyset$  then
2:   if  $u.vector \cdot \widehat{V}_Q > \tau$  then
3:     if  $u$  is not a leaf node then
4:        $DepthFirstSearch(u.lp, \widehat{V}_Q, k, \tau, NS)$ ;
5:        $DepthFirstSearch(u.rp, \widehat{V}_Q, k, \tau, NS)$ ;
6:     else if  $u$  is a leaf node then
7:       Add  $u$  to  $NS$ ;
8:       if  $|NS| > k$  then
9:         Delete the leaf node from  $NS$  that has the lowest semantic relevance score to  $\widehat{V}_Q$ ;
10:      end if
11:      if  $|NS| = k$  then
12:         $\tau = \min\{x.vector \cdot \widehat{V}_Q | x \in NS\}$ , where  $\min\{*\}$  is to get the minimum of a item set;
13:      end if
14:    end if
15:  end if
16: end if

```

and $NS = \{u_{4,2}, u_{4,1}\}$. Then, the search processing gets to the leaf node $u_{3,2}$ through the green path. Because the inner product between $\widehat{V}_Q$ and the vector of $u_{3,2}$ is larger than that between $\widehat{V}_Q$ and the vector of $u_{4,1}$, $u_{4,1}$ is removed from NS while $u_{3,2}$ is added into NS , and τ is updated where $\tau = u_{4,2}.vector \cdot \widehat{V}_Q = 0.08$ and $NS = \{u_{3,2}, u_{4,2}\}$. When the search gets to the node $u_{2,2}$ through the yellow path, because the inner product between $\widehat{V}_Q$ and the vector of $u_{2,2}$ is smaller than τ , the subtree rooted by this node are pruned and the search processing is finished. Finally, the search result is the set of encrypted documents corresponding to the leaf nodes of NS , i.e., $R = \{\widehat{f}_5, \widehat{f}_3\}$.

5 Threshold presetting-based optimization on EPSAS

As shown in the search processing algorithms (Algorithm 2 and 3), the pruning threshold τ can improve the search efficiency. Any subtrees, the inner products between the vectors of whose root nodes and a search trapdoor vector are less than τ , are directly pruned without subsequent search. However, τ is set to 0 at the beginning and it gradually grows up as more leaf nodes are traversed. It means that the subtree pruning is rarely effective at the beginning. If a proper value can be chosen to assign to τ when a search is started, then the search processing will be further improved in efficiency. Driven by this idea, we present a threshold presetting-based optimization on the proposed EPSAS scheme in this section.

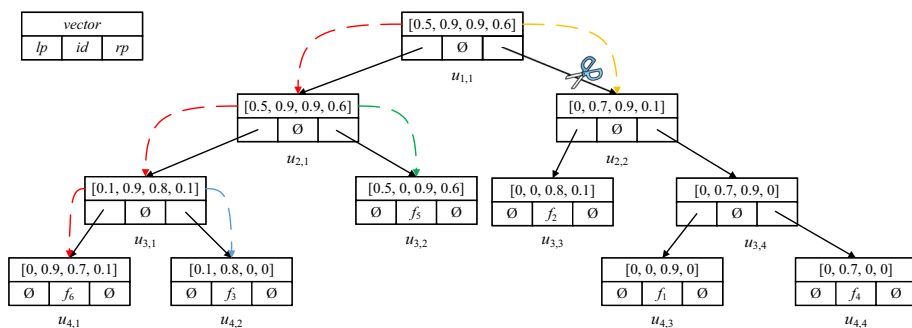


Figure 4 An example of BCI-tree-based search

To describe the threshold presetting-based optimization, we present a threshold referring matrix which is a novel designed structure to provide proper pruning thresholds for search processing.

Definition 4 Threshold referring matrix (TR-matrix) A threshold referring matrix, denoted as G , is an $h \times l$ -dimensional matrix, where h is the number of keywords of dictionary, l is the referring depth parameter, and $k < l < n$.

- The item $G[i][j]$ records a noise-obfuscated relevance score between the keyword w_i and a document in F .
- The items stored in each row of G are in descending order, i.e.,

$$\forall i \in \{1, 2, \dots, h\}, p, q \in \{1, 2, \dots, l\}, p < q \rightarrow G_{i,p} > G_{i,q}. \quad (17)$$

On the basis of the TR-matrix, we present optimizations in the data preprocessing module and semantic-aware search module to show the optimized EPSAS. It should be noted that the index building module remains unchanged.

Optimization in the data preprocessing module We add an algorithm after *LDATrain* to generate the TR-matrix, which is named *GenTRM* and provides proper candidate thresholds for presetting the pruning thresholds in search processing. Other algorithms in the data preprocessing module, such as *Genkey*, *LDATrain* and *EncDocs*, are all kept the same.

$G \leftarrow \text{GenTRM}(W, \Gamma, \Psi, P_T, P_W, I)$ The *GenTRM* algorithm is to generate the TR-matrix G by the data owner, which is shown as the following steps.

- 1) For each $w_i \in W$, the data owner computes the single keyword topic vector $V_{w_i} = \frac{P_T \circ \Psi[i]}{P_W[i]}$, where $V_{w_i}[j]$ indicates the probability that the topic intent of w_i belongs to the topic t_j . Afterwards, the data owner computes a list of inner products for w_i , $L_{w_i} = \{V_{w_i} \cdot \Gamma[j] | j \in \{1, 2, \dots, n\}\}$, which records the semantic relevance scores between w_i and every documents in F . Then, the data owner sorts L_{w_i} in descending order, and copies the former l values into the i th row of TR-matrix G .
- 2) The data owner copies all items of G into a temporary list L_t and then sorts L_t in descending order. Afterwards, the data owner computes the gaps of every two neighboring items in L_t to obtain the noise parameter σ that is the minimum of the gaps, i.e., $\sigma = \min\{L_t[i] - L_t[i + 1] | i \in \{1, 2, \dots, h \times l - 1\}\}$

- 3) For each item $G[i][j] \in G$, the data owner sets $G[i][j] = G[i][j] - rand(0, \sigma)$ where $rand(0, \sigma)$ is to get a random value between 0 and σ . At this time, the TR-matrix G has been setup and is transmitted to the data user for presetting proper pruning thresholds when starting searches.

As shown in the algorithm *GenTRM*, each value stored in G is a semantic relevance score between a keyword and a document at the beginning. To hide the plaintext information of semantic relevance scores, each value in G is subtracted by a noise value generated by $rand(0, \sigma)$. Since the random generated noises are positive and smaller than the minimum gap σ , the order of the values in G remains unchanged after the noise addition.

Optimization in the semantic-aware search module We add an algorithm after *GenTrapdoor* to preset the pruning threshold τ , which is named *PresetTau* and executed by the data user.

$\tau \leftarrow \text{PresetTau}(G, k, Q)$ The *PresetTau* algorithm is run by the data user to preset the initial value of the pruning threshold τ when a search is started. To compute the initial value of τ , the TR-matrix G , the number of requested documents k and the searched keywords are taking as the input, and the computation follows (18).

$$\tau = \frac{\max\{G[i][k] | w_i \in Q\}}{|Q|} \quad (18)$$

When a search is started by the data user, the *GenTrapdoor* algorithm is run by the data user to generate the search trapdoor, which is kept the same as that in EPSAS. Then, the data user uses the *PresetTau* algorithm to preset the initial value of the pruning threshold τ . Afterwards, the data user transmits the search trapdoor and the preset τ to the cloud server. And the cloud server uses the preset τ instead of $\tau = 0$ to run the *SASearch* algorithm. Since the pruning threshold τ has preset value, the subtree filtering mechanism works even at the beginning of search processing, and thus the search speed is improved. To validate the effectiveness of such threshold presetting-based optimization on EPSAS, we will implement the baseline version and the optimized version of EPSAS and compare them in search efficiency.

6 Security analysis

In the proposed EPSAS scheme, the privacy of documents, BCI-tree index, queried keywords, trapdoor unlinkability and keyword information are preserved from the curious cloud server under the “honest-but-curious” threat model.

Theorem 1 *The privacy of documents is preserved in EPSAS.*

Proof According to the search algorithm in the proposed schemes, the documents themselves do not participate in the search processing and they are encrypted by secure symmetric encryption (such as Blowfish, AES, etc.). Because the secure key K is only shared between the data owner and the data user but the cloud server has no idea of it, it is computation infeasible for the cloud server to obtain the plaintext documents, and EPSAS is CPA-secure for documents. $\square$

Theorem 2 *The privacy of BCI-tree index and queried keywords are preserved in EPSAS.*

Proof For the BCI-tree index and queried keywords in our schemes, they are both in the form of ciphertext to be accessed by the cloud server. The sensitive information protected

in the ciphertext are vectors, such as the encrypted document topic vectors in the BCI-tree index $\widehat{\mathcal{I}}$ and query topic vectors in trapdoors. Since the cloud server can only access the encrypted index and trapdoors, it could attempt to obtain the plaintext vectors in the indexes and trapdoor in the known ciphertext threat model and the known background threat model. According to the procedures of the *BuildIndex* algorithm, the data owner splits each vector V into $\{V', V''\}$ depending on the random generated m -bit vector S which is unknown to the cloud server. Then, the data owner generates the encrypted vector $\widehat{V} = \{\widehat{V}', \widehat{V}''\}$ according to (11). Thus, we have two equations as shown in (19).

$$\begin{cases} V' M_1^{-1} = \widehat{V}' \\ V'' M_2^{-1} = \widehat{V}'' \end{cases} \quad (19)$$

In (19), V' , V'' , $\widehat{V}'$, and $\widehat{V}''$ are all m -dimensional vectors, and M_1 and M_2 are both $m \times m$ -dimensional matrices. The cloud server can only access $\widehat{V}'$, and $\widehat{V}''$ but has no idea of V' , V'' , M_1 and M_2 . Thus, for the cloud server, there are $2m$ quadratic equations with $2m^2 + 2m$ unknowns and $2m$ knowns in (19), and it is computational infeasible to solve those equations to obtain the plaintext of V' , V'' , M_1 , M_2 and V . Because the generation of the search trapdoor is the same as the encrypted vector $\widehat{V}$, the queried keywords are also protected in private. As a result, the proposed EPSAS scheme is able to protect the privacy of BCI-tree index and queried keywords in the known ciphertext threat model and the known background threat model. $\square$

Theorem 3 *The trapdoor unlinkability is preserved in EPSAS.*

Proof In EPSAS, for a search Q having multiple queried keywords, the corresponding query topic vector is randomly split into two vectors by the m -bit random vector S according to (13). The generated vectors after splitting are then encrypted by applying inner products with two random generated invertible matrices M_1 and M_2 . The trapdoor for Q is the encrypted vectors. Even for the same queried keywords in different searches, the generated trapdoors are different. Therefore, the link between the generated trapdoors of the same searches is break and the trapdoor unlinkability is guaranteed. However, the cloud server can infer whether the search requests are the same from the same search paths and the same semantic relevance scores. In order to solve this problem, phantom terms [38, 41] can be added to provide different search paths and semantic relevance scores with the same search requests, but the precision of search result will decrease. $\square$

Theorem 4 *The privacy of keywords is preserved in EPSAS.*

Proof As shown in Theorem 2, the privacy of BCI-tree index and queried keywords are well preserved in EPSAS. According to the *LDATrain* algorithm, the information of keywords is embedded in the keyword-topic correlation matrix Ψ which is only shared between the data owner and the data user but the cloud server have no idea about it. Thus, the privacy of keywords is protected in the known ciphertext threat model. But in the TF-IDF model-based search schemes [5, 34, 38, 41], document vectors are represented by the term frequency (TF) values of keywords while query vectors are represented the inverse document frequency (IDF) values of queried keywords. In the known background threat model, the cloud server is assumed to know more knowledge, such as the term frequency statistics of keywords. By using such information, the cloud server could perform TF value statistical attacks to deduce specific keywords [38, 44]. However, the proposed EPSAS scheme adopts the LDA model to generate topic relevance-based vectors rather than the term frequency-based vectors to

demonstrate documents. The topic relevance does not leak private information on keywords in the known background threat model. The reason is that the topic relevance values in document vectors represent the semantic relevance in the concept of topics which are irrelevant to the term frequency information of keywords, and the cloud server cannot deduce any keywords by conducting TF value statistical attacks. Besides, the data outsourced by the data owner has nothing to do with the information of keywords, so the cloud server cannot obtain any information about keywords. As a result, the privacy of keywords is preserved from the curious cloud server.

In the known background threat model, the cloud server could conduct statistical attack to obtain topic information about keywords or documents. However, the topics in the LDA topic model are manually set and have no specific meanings. Keywords with higher probabilities in the same topic are semantically related but the topic itself has no specific meaning. Even if a keyword and a document are highly related in topic distribution, the relationship between the keyword and the document cannot be directly deduced from topic information, because the keyword may not appear in the document. In order to prevent topic information leakage, phantom terms can be added to disrupt the relevance score and defend the statistical attack. $\square$

7 Performance evaluation

In this section, we perform evaluation on the proposed EPSAS scheme and its optimized version which is denoted as EPSAS+. We compare the experimental result with that in TFIDF-MRSE [41] and LDA-EMRSE [10]. We implement four schemes by using Python 3.9 in Windows 10 with Intel Core(TM) i7-11800H. The evaluation is based on two metrics, the semantic precision of search result and the search time cost. The dataset used in the evaluation is 20 newsgroups [23] which is a popular data set for experiments in text applications. The data set has a set of approximately 20,000 news documents and is partitioned into 20 different newsgroups, each of which corresponds to a different topic. Documents in the same newsgroup are very closely related to each other in topic while documents in different newsgroups are nearly unrelated in topic. In this evaluation, the default setting of the quantity of documents n , the quantity of topics m , the number of queried keywords $|Q|$ and the number of requested documents k are $n = 8000$, $m = 100$, $|Q| = 3$ and $k = 100$, respectively.

7.1 Semantic precision evaluation

To evaluate the semantic precision of search result, we adopt the definition of the semantic precision from [18]. Assuming that the queried keywords belong to the same category, if a document belongs to different categories, then the queried keywords and the document are irrelevant in semantic. The calculation of the semantic precision is shown in (20),

$$Precision = \frac{TP}{TP + FP} \times 100\%, \quad (20)$$

where TP is the quantity of documents of search result in the same category of queried keywords, and FP is the quantity of documents of search result in the different category of queried keywords.

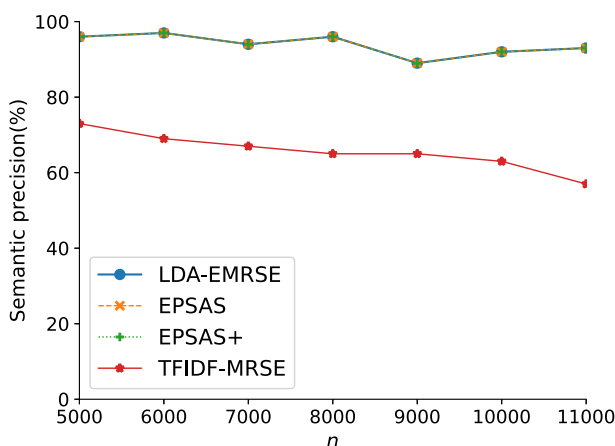


Figure 5 Semantic precision versus n

In this experiment, we evaluate the effect of parameters n , k and $|Q|$ on the semantic precision of EPSAS, EPSAS+, TFIDF-MRSE and LDA-EMRSE. The experimental result is presented in Figures 5 ~ 7

- Figure 5 indicates that as the quantity of documents n grows up, the semantic precision of EPSAS, EPSAS+ and LDA-EMRSE remains almost stable, while the semantic precision of TFIDF-MRSE gradually decreases. The reason is that in EPSAS, EPSAS+ and LDA-EMRSE all adopt LDA model. As n increases, the topics extracted by the LDA model can better reflect the semantic features of the documents and slightly affect the semantic precision of search results. But in TFIDF-MRSE, documents in search result are determined by the frequency statistics of keyword matches, which does not consider the inherent semantics of documents and keywords. The more documents are searched, the higher the probability of semantic deviation between search results and queried keywords becomes greater, and the lower semantic precision of search results.

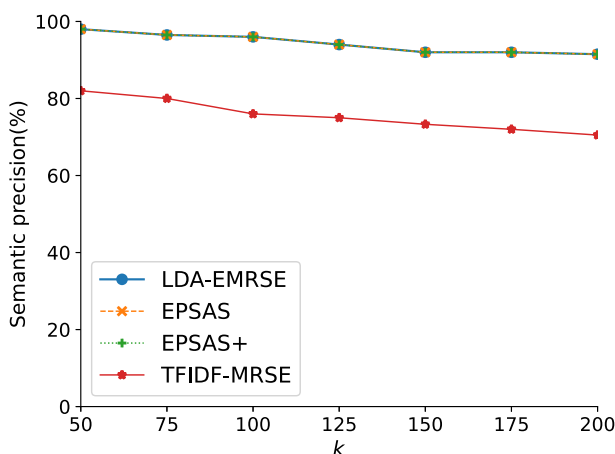


Figure 6 Semantic precision versus k

- 2) Figure 6 indicates that as the quantity of requested documents k increases, the semantic precision of the four schemes all have a little decrements. The reason is that as the quantity of requested documents increases, documents with lower semantic relevance scores to the queried keywords are appended to the search result, which lead to a little decrease in the semantic precision of search result.
- 3) Figure 7 indicates that as the quantity of queried keywords $|Q|$ grows up, the semantic precision of the four schemes all increase at first and then turn to be stable. The reason is that as more keywords are queried, the search intent of the data user becomes more explicit in semantic and the documents in search results can more precisely represent the latent semantics of the search, and thus, the semantic precision increases. However, when the growth of queried keywords reaches a certain level, the semantic feature reflected by the queried keywords tends to be stable and the semantic precision of search result also turns to stable.

In addition, Figures 5 ~ 7 all show that the semantic precision of EPSAS, EPSAS+ and LDA-EMRSE are the same and higher than that of TFIDF-MRSE. The reason is that EPSAS, EPSAS+ and LDA-EMRSE are all based on the LDA model and have the ability to find the same result, thus they have the same semantic precision. At the same time, EPSAS, EPSAS+ and LDA-EMRSE use the LDA model to generate semantic information-embedded vectors for documents while TFIDF-MRSE uses the TF-IDF model to generate term frequency statistic-based document vectors. The former three schemes can better reflect the latent semantic information of documents and keywords, so the search results of the former three schemes have higher semantic precision.

7.2 Search time cost evaluation

In this section, we evaluate the search time cost of EPSAS, EPSAS+, TFIDF-MRSE and LDA-EMRSE. The experimental result is shown in Figures 8 ~ 10.

- 1) Figure 8 shows that as the quantity of documents n increases, the search time cost of four schemes all gradually increase. The reason is that four schemes all need to traverse

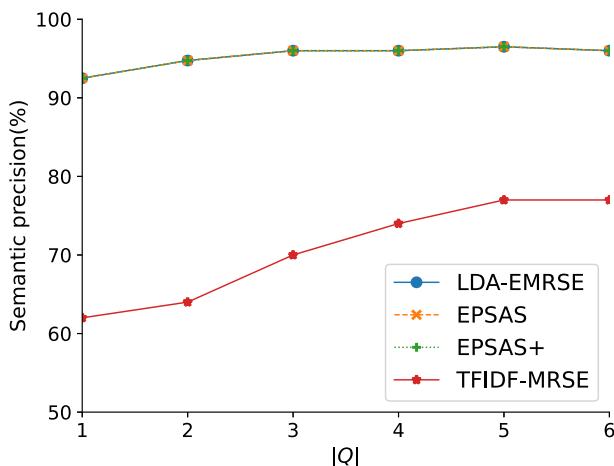


Figure 7 Semantic precision versus $|Q|$

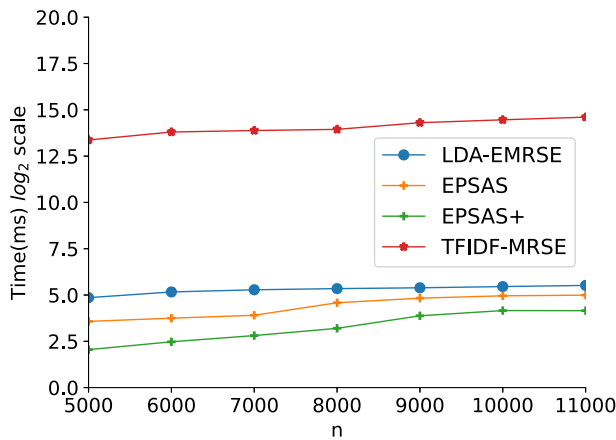


Figure 8 Search time cost versus n

nodes of the tree-based indexes and perform inner products to calculate search results. The larger n is, the more nodes need to be traversed, so the number of inner products increases which causes the increment of search time cost.

- 2) Figure 9 indicates that as the quantity of requested documents k increases, the search time cost of four schemes also all gradually increase. The reason is that with the increment of k , the search processing of four schemes all need to traverse more nodes in their tree-based indexes, respectively. It means that more inner products need to be calculated, which costs more search time.
- 3) Figure 10 indicates that as the quantity of queried keywords $|Q|$ increases, the search time costs of four schemes are almost stable, which means that $|Q|$ has no obvious effect on search time cost. The reason is that in four schemes, no matter how many queried keywords are there, they are all converted into fixed dimensional vectors, which does not cause significant changes in search time cost.

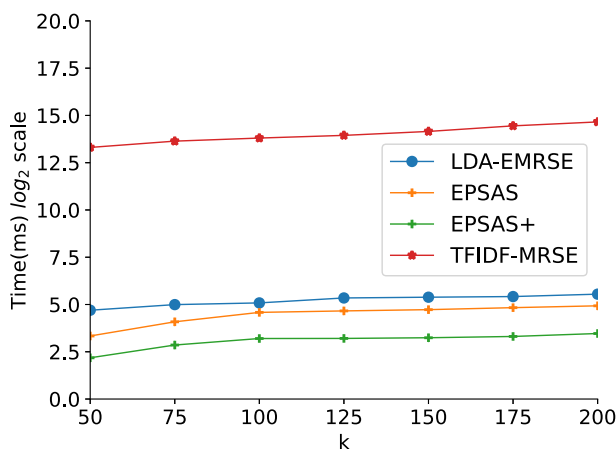


Figure 9 Search time cost versus k

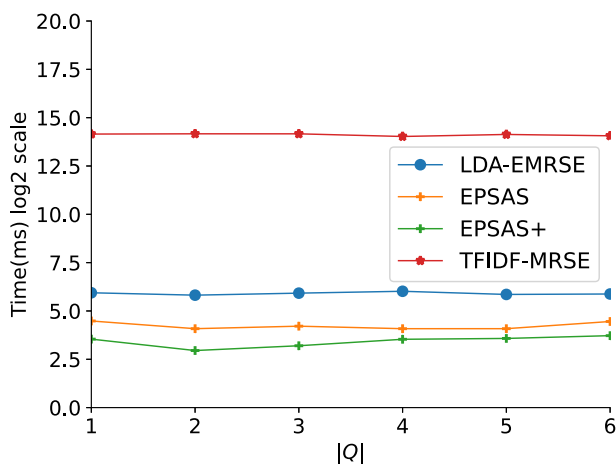


Figure 10 Search time cost versus $|Q|$

The experimental results in Figures 8 ~ 10 all show that the search time cost of TFIDF-MRSE is much higher than the other three schemes. The reason is that the dimension of vectors in TFIDF-MRSE is the same as the quantity of keywords in the dictionary. It is significantly higher than the dimension of vectors adopted in EPSAS, EPSAS+ and LDA-EMRSE, which equals to the quantity of topics used in the LDA model. The search time cost of an inner product between vectors with small dimension is obviously faster than that with large dimension.

Meanwhile, EPSAS, EPSAS+ and LDA-EMRSE are all semantic-aware search schemes. In these three semantic-aware schemes, the former two schemes is more efficient than the last scheme in search time cost. The reason is that these schemes all use binary tree-based indexes to perform search processing. In LDA-EMRSE, documents are randomly deployed to leaf nodes of its index. But in EPSAS and EPSAS+, documents are well organized in the leaf nodes of the indexes and documents with high semantic relevance are usually near with each other. Such document deployment strategy can reduce the quantity of nodes to be traversed during the search processing in EPSAS and EPSAS+, which is beneficial to improve the search efficiency. In addition, EPSAS+ adopts the pruning threshold presetting method which helps to filter out those unqualified subtrees more efficiently in search processing. It further speeds up the search processing and makes EPSAS+ the best of these four schemes in search time cost.

8 Conclusion

The searchable encryption in the cloud environment is an effective method to provide multi-keyword searches while preserve the privacy of the outsourced data. An efficient privacy-preserving semantic-aware multi-keyword search scheme is proposed in this paper. The scheme constructs an novel BCI-tree-based index by using the LDA model and the divisive hierarchical clustering algorithm. A depth-first search algorithm is designed to perform efficient multi-keyword search processing. To further improve the efficiency of search

processing, a threshold presetting-based optimization is proposed. The experimental result indicates that the proposed scheme has better performance in terms of semantic precision and search time cost.

Acknowledgements Not applicable.

Author Contributions This work thanks the following authors for their contributions: Qian Zhou and Hua Dai contributed to the conception of the study; Qian Zhou and Yuanlong Liu performed the experiment; Qian Zhou and Zheng Hu contributed significantly to security analysis and manuscript preparation; Qian Zhou and Hua Dai performed the data analyses and wrote the manuscript; Geng Yang and Xun Yi helped perform the analysis with constructive discussions.

Funding This work was supported by the National Natural Science Foundation of China under the grant Nos.61902199, 61872197, and 61972209; and the Jiangsu Province Postgraduate Scientific Research Innovation Program under the grand No. KYCX22_0984.

Data Availability Details of the datasets have been described in Section 8.

Declarations

Ethical Approval and Consent to participate Our manuscripts were not submitted to multiple journals for simultaneous consideration and original. All authors agree with the content of the article.

Human and Animal Ethics Not applicable.

Consent for publication Our manuscript is approved by all authors for publication.

Competing interests No conflict of interest exists in the submission of this manuscript, and manuscript is approved by all authors for publication.

References

1. Ballard, L., Kamara, S., Monrose, F.: Achieving efficient conjunctive keyword searches over encrypted data. In: Proceedings of 2005 International Conference on Information and Communications Security. pp. 414–426. Springer (2005)
2. Blei, D.M., Ng, A.Y., Jordan, M.I.: Latent dirichlet allocation. *J. Mach. Learn. Res.* **3**(1), 993–1022 (2003)
3. Boneh, D., Waters, B.: Conjunctive, subset, and range queries on encrypted data. In: Theory of Cryptography Conference. pp. 535–554. Springer (2007)
4. Cao, N., Wang, C., Li, M., Ren, K., Lou, W.: Privacy-preserving multi-keyword ranked search over encrypted cloud data. In: Proceeding of the 30th IEEE International Conference on Computer Communications. pp. 829–837. IEEE (2011)
5. Cao, N., Wang, C., Li, M., Ren, K., Lou, W.: Privacy-preserving multi-keyword ranked search over encrypted cloud data. *IEEE Trans. Parallel Distrib. Sys.* **25**(1), 222–233 (2014)
6. Cao, Q., Li, Y., Wu, Z., Miao, Y., Liu, J.: Privacy-preserving conjunctive keyword search on encrypted data with enhanced fine-grained access control. *World Wide Web* **23**, 959–989 (2020)
7. Chang, Y.C., Mitzenmacher, M.: Privacy preserving keyword searches on remote encrypted data. In: Proceedings of 2005 International Conference on Applied Cryptography and Network Security. pp. 442–455. Springer (2005)
8. Chen, C., Zhu, X., Shen, P., Hu, J., Guo, S., Tari, Z., Zomaya, A.Y.: An efficient privacy-preserving ranked keyword search method. *IEEE Trans. Parallel Distrib. Syst.* **27**(4), 951–963 (2016)
9. Curtmola, R., Garay, J., Kamara, S., Ostrovsky, R.: Searchable symmetric encryption: improved definitions and efficient constructions. *J. Comput. Secur.* **19**(5), 895–934 (2011)
10. Dai, H., Dai, X., Yi, X., Yang, G., Huang, H.: Semantic-aware multi-keyword ranked search scheme over encrypted cloud data. *J. Netw. Comput. Appl.* **147**, 102442 (2019)

11. Dai, H., Yang, M., Yang, G., Xiang, Y., Hu, Z., Wang, H.: A keyword-grouping inverted index based multi-keyword ranked search scheme over encrypted cloud data. *IEEE Trans. Sustain. Comput.* **7**(3), 561–578 (2022)
12. Dai, X., Dai, H., Rong, C., Yang, G., Xiao, F.: Enhanced semantic-aware multi-keyword ranked search scheme over encrypted cloud data. *IEEE Trans. Cloud Comput.*, pp. 1–16 (2020)
13. Delfs, H., Knebl, H.: Introduction to cryptography principles and applications, 3rd edn. Springer-Verlag, Berlin (2007)
14. Fu, Z., Huang, F., Ren, K., Weng, J., Wang, C.: Privacy-preserving smart semantic search based on conceptual graphs over encrypted outsourced data. *IEEE Trans. Inf. Forensics Sec.* **12**(8), 1874–1884 (2017)
15. Fu, Z., Sun, X., Ji, S., Xie, G.: Towards efficient content-aware search over encrypted outsourced data in cloud. In: Proceedings of the 35th Annual IEEE International Conference on Computer Communications. pp. 1–9. IEEE (2016)
16. Fu, Z., Sun, X., Linge, N., Zhou, L.: Achieving effective cloud search services: multi-keyword ranked search over encrypted cloud data supporting synonym query. *IEEE Trans. Cons. Electr.* **60**(1), 164–172 (2014)
17. Fu, Z., Xia, L., Sun, X., Liu, A.X., Xie, G.: Semantic-aware searching over encrypted data for cloud computing. *IEEE Trans. Inf. Forensics Sec.* **13**(9), 2359–2371 (2018)
18. Gabryel, M., Damaševičius, R., Przybyszewski, K.: Application of the bag-of-words algorithm in classification the quality of sales leads. In: Proceedings of 2018 International Conference on Artificial Intelligence and Soft Computing. pp. 615–622. Springer (2018)
19. Hozhabr, M., Asghari, P., Javadi, H.: Dynamic secure multi-keyword ranked search over encrypted cloud data. *J. Inf. Sec. Appl.* **61**(1), 1–12 (2021)
20. Hua, J., Liu, Y., Chen, H., Tian, X., Jin, C.: An enhanced wildcard-based fuzzy searching scheme in encrypted databases. *World Wide Web* **23**, 2185–2214 (2020)
21. Ibrahim, A., Jin, H., Yassin, A.A., Zou, D.: Secure rank-ordered search of multi-keyword trapdoor over encrypted cloud data. In: Proceedings of 2012 IEEE Asia-Pacific Services Computing Conference. pp. 263–270. IEEE (2012)
22. Kiayias, A., Oksuz, O., Russell, A., Tang, Q., Wang, B.: Efficient encrypted keyword search for multi-user data sharing. In: Proceedings of European Symposium on Research in Computer Security. pp. 173–195. Springer (2016)
23. Lang, K.: Newsweeder: Learning to filter netnews. In: Proceedings of the 12th International Conference on Machine Learning. pp. 331–339. Elsevier (1995)
24. Li, J., Wang, Q., Wang, C., Cao, N., Ren, K., Lou, W.: Fuzzy keyword search over encrypted data in cloud computing. In: Proceedings of the 29th IEEE International Conference on Computer Communications. pp. 1–5. IEEE (2010)
25. Liang, Y., Li, Y., Zhang, K., Ma, L.: DMSE: dynamic multi-keyword search encryption based on inverted index. *J. Syst. Architect.* **119**, 1–10 (2021)
26. Liu, C., Zhu, L., Chen, J.: Efficient searchable symmetric encryption for storing multiple source dynamic social data on cloud. *J. Netw. Comput. Appl.* **86**, 3–14 (2017)
27. Liu, Q., Peng, Y., Wu, J., Wang, T., Wang, G.: Secure multi-keyword fuzzy searches with enhanced service quality in cloud computing. *IEEE Trans. Network and Service Management* **18**(2), 2046–2062 (2020)
28. Orencik, C., Kantarcioglu, M., Savas, E.: A practical and secure multi-keyword search method over encrypted cloud data. In: Proceedings of IEEE 16th International Conference on Cloud Computing. pp. 390–397. IEEE (2013)
29. Poon, H.T., Miri, A.: An efficient conjunctive keyword and phase search scheme for encrypted cloud storage systems. In: Proceedings of 2015 IEEE 8th International Conference on Cloud Computing. pp. 508–515. IEEE (2015)
30. Roux, M.: A comparative study of divisive and agglomerative hierarchical clustering algorithms. *J. Class.* **35**(2), 345–366 (2018)
31. Scheuermann, P., Ouksel, M.: Multidimensional b-trees for associative searching in database systems. *Inf. Syst.* **7**(2), 123–137 (1982)
32. Song, D.X., Wagner, D., Perrig, A.: Practical techniques for searches on encrypted data. In: Proceeding of 2000 IEEE Symposium on Security and Privacy. pp. 44–55. IEEE (2000)
33. Sun, W., Liu, X., Lou, W., Hou, Y.T., Li, H.: Catch you if you lie to me: efficient verifiable conjunctive keyword search over large dynamic encrypted cloud data. In: Proceedings of the 34th IEEE International Conference on Computer Communications. pp. 2110–2118. IEEE (2015)
34. Sun, W., Wang, B., Cao, N., Li, M., Lou, W., Hou, Y.T., Li, H.: Verifiable privacy-preserving multi-keyword text search in the cloud supporting similarity-based ranking. *IEEE Trans. Parallel Distrib. Syst.* **11**(25), 3025–3035 (2014)

35. Swaminathan, A., Mao, Y., Su, G.M., Gou, H., Varna, A.L., He, S., Wu, M., Oard, D.W.: Confidentiality-preserving rank-ordered search. In: Proceedings of the 2007 ACM workshop on Storage security and survivability. pp. 7–12. ACM (2007)
36. Tseng, C.Y., Lu, C., Chou, C.F.: Efficient privacy-preserving multi-keyword ranked search utilizing document replication and partition. In: Proceedings of the 12th Annual IEEE Consumer Communications and Networking Conference (CCNC). pp. 671–676. IEEE (2015)
37. Wang, B., Yu, S., Lou, W., Hou, Y.T.: Privacy-preserving multi-keyword fuzzy search over encrypted data in the cloud. In: Proceedings of the 33th IEEE International Conference on Computer Communications. pp. 2112–2120. IEEE (2014)
38. Wang, C., Cao, N., Ren, K., Lou, W.: Enabling secure and efficient ranked keyword search over outsourced cloud data. *IEEE Trans. Parallel Distrib. Syst.* **23**(8), 1467–1479 (2011)
39. Wang, C., Ren, K., Yu, S., Urs, K.M.R.: Achieving usable and privacy-assured similarity search over outsourced cloud data. In: Proceedings of the 31th IEEE International Conference on Computer Communications. pp. 451–459. IEEE (2012)
40. Wang, P., Ravishankar, C.V.: On masking topical intent in keyword search. In: Proceedings of 2014 IEEE 30th International Conference on Data Engineering. pp. 256–267. IEEE (2014)
41. Xia, Z., Wang, X., Sun, X., Wang, Q.: A secure and dynamic multi-keyword ranked search scheme over encrypted cloud data. *IEEE Trans. Parallel Distrib. Syst.* **27**(2), 340–352 (2016)
42. Xia, Z., Zhu, Y., Sun, X., Chen, L.: Secure semantic expansion based search over encrypted cloud data supporting similarity ranking. *J. Cloud Comput.* **3**(1), 1–11 (2014)
43. Zhu, X., Dai, H., Yi, X., Yang, G., Li, X.: Muse: an efficient and accurate verifiable privacy-preserving multikeyword text search over encrypted cloud data. *Secur. Commun. Netw.* **2017**, 1–17 (2017)
44. Zerr, S., Olmedilla, D., Nejdl, W., Siberski, W.: Zerber+: Top-k retrieval from a confidential index. In: Proceedings of the 12th International Conference on Extending Database Technology: Advances in Database Technology. pp. 439–449 ACM (2009)
45. Zhang, B., Zhang, F.: An efficient public key encryption with conjunctive-subset keywords search. *J. Netw. Comput. Appl.* **34**(1), 262–267 (2011)
46. Zhang, W., Xiao, S., Lin, Y., Zhou, T., Zhou, S.: Secure ranked multi-keyword search for multiple data owners in cloud computing. In: 2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks. pp. 276–286. IEEE (2014)
47. Zhou, Q., Dai, H., Shen, W., Liu, Y., Yang, G.: Evss: An efficient verifiable search scheme over encrypted cloud data. *World Wide Web* pp. 1–21 (2022)
48. Zhu, X., Liu, Q., Wang, G.: A novel verifiable and dynamic fuzzy keyword search scheme over encrypted data in cloud computing. In: Proceedings of 2016 IEEE Trustcom/BigDataSE/ISPA. pp. 845–851. IEEE (2016)
49. Zhou Q., Dai H., Hu Z., Liu Y., Yang G.: SAPMS: A Semantic-aware Privacy-preserving Multi-keyword Search Scheme in Cloud. In: Proceedings of the 6th APWeb-WAIM International Joint Conference on Web and Big Data. pp. 251–263. LNCS (2022)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.